browser Control

Browser controls are the commands or methods used to programmatically control and interact with the browser during automation testing. In Playwright they are available through the Browser, BrowserContext, and Page fixture.

1)page.setViewPortSize()=it is a Playwright method that sets the width and height of the browser window (viewport) in pixels

```
syntax:await page.setViewportSize({ width: value, height: value });
```

2)pageviewportSize()=it is a Playwright method that returns the current width and height of the browser viewport. It only reads the size,it does not change it.

```
syntax:const size = page.viewportSize();
       console.log(size);
```

3)page.title()=it is a Playwright method that returns the title of the current web page

```
syntax:const title = await page.title();
       console.log(title);
```

4)page.url()=it is a Playwright method that returns the URL (web address) of the current page.

```
syntax:const url = page.url();
       console.log(url);
```

5)page.goto()=it is a Playwright method that opens a URL (web address) in the browser page

```
syntax:await page.goto('url');
```

6)context.cookies()=returns all the cookies stored in the current browser context.

```
syntax:const cookies = await context.cookies();
       console.log(cookies);
```

7)chromium.launch() = opens a new Chromium browser (the engine behind Chrome) so your Playwright script can control it.

```
syntax:const browser = await chromium.launch();
```

8)browser.newContext() = it creates a new, isolated browser context (like a fresh incognito session) inside a browser.

```
syntax:const context = await browser.newContext();
```

9)context.newPage() =opens a new tab (page) inside a browser context.

syntax:const page = await context.newPage();

10)page.screenshot()=takes a screenshot of the current page and saves it as an image file.

syntax:await page.screenshot({ path:'example/page.png'});

```
import {chromium, test} from "@playwright/test"
test.skip("browserControlsMethod",async({page})=>{
await page.goto("https://www
let size= page.viewportSize(
console.log(size)
await page.setViewportSize({width:800,height:500});
await page.waitForTimeout(3000)
let size1= page.viewportSize();
console.log(size1)
let titleofapp:string=await page.title();
let urlofapp:string= page.url();
console.log(titleofapp)
console.log(urlofapp)
})
Run | Debug | Inspect
test("contextcookies",async({browser})=>{
const context1= await browser.newContext();
const page1 =await context1.newPage();
await page1.goto("https://www.facebook.com/")
console.log(await context1.cookies())
await page1.waitForTimeout(3000)

const context2= await browser.newContext();
const page2 =await context2.newPage();
await page2.goto("https://demowebshop.tricentis.com/")
console.log(await context2.cookies())
await page2.waitForTimeout(3000)
})
Run | Debug | Inspect
test("browserinstance",async()=>{
    let browser =await chromium.launch()
    let context=await browser.newContext()
    let page=await context.newPage()
    await page.goto("https://www.facebook.com/")
    await page.waitForTimeout(3000)
})
Run | Debug | Inspect
test.only("screenshot",async({page})=>{
  await  page.goto("https://www.facebook.com/")
  // await page.screenshot({path:"screenshot/facebookscreenshot.png"})
    await page.waitForTimeout(3000)
   let time= new Date().getTime();
 await page.screenshot({path:`screenshot/facebookscreenshot-${time}.png`})
})
```

(property) width: number

page width in pixels.